

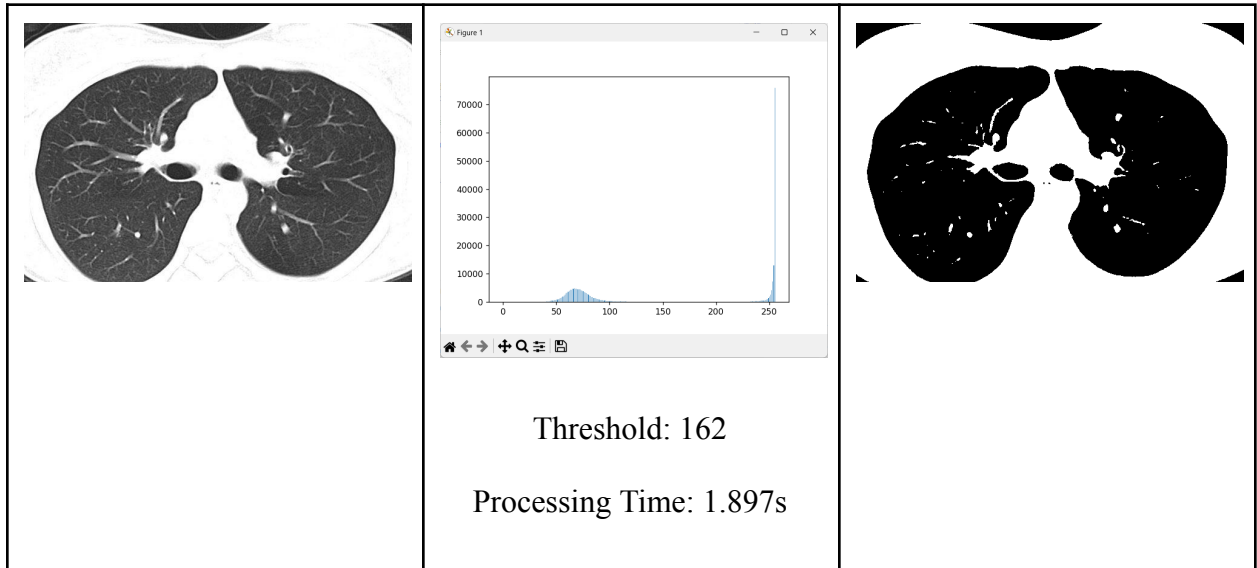
Abstract

In this lab, I used a recursive version of Otsu's Method to automate threshold selection for image binarization. Using a histogram of an input image's intensity, my program iterates through possible threshold values and calculates the between class variance of the segmented classes. If the variance is still increasing, the function is called again for the next intensity. If not, the maximum variance has been found and binarization is performed at that threshold.

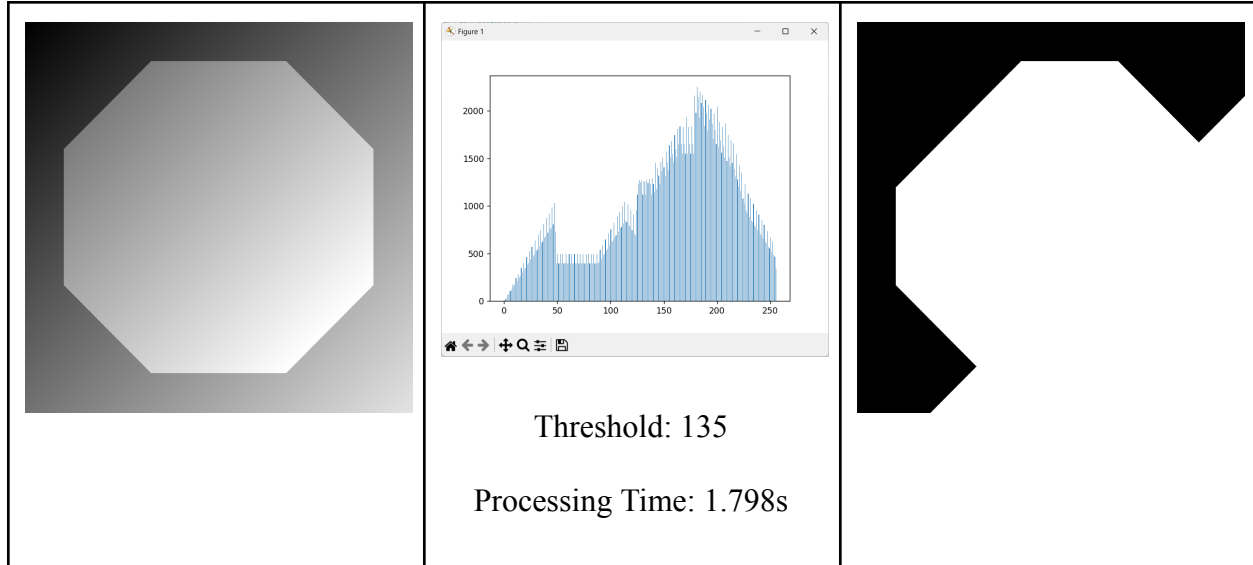
Introduction

Three input images were binarized using the threshold suggested by Otsu's Method. The first is a grayscale image of lungs, the second is a pre-binarized octagon with a gradient overlaid, and the final is an RGB image of a lake. These experiments demonstrate the situational effectiveness of Otsu's Method in segmentation, particularly when limited to a single color channel.

Experiments and Results



This first image represents the optimal circumstances for my implementation of Otsu's Method. The image is grayscale without significant noise, and the two regions already fall on opposite sides of the color spectrum. As such, the results are perfectly satisfactory. The only area for improvement would be the addition of spatial processing to filter out the brighter bronchi.

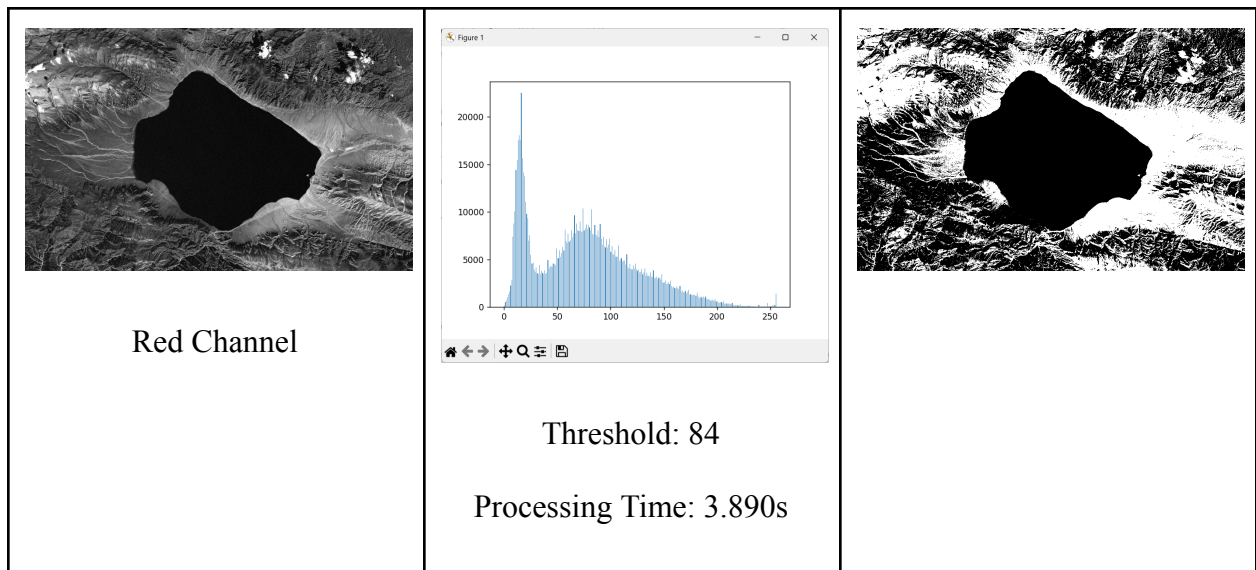


The second image is somewhat misleading, as the human eye sees two wholly distinct classes. This is not captured in the intensity histogram, though, which is how Otsu's Method "sees" the image. The overlaid gradient dramatically changes the "light" and "dark" values in different regions, making them overlap on the intensity histogram. No possible threshold could segment out the octagon. Accordingly, the results in this case are unsatisfactory.

The binarization performed correctly for about $\frac{1}{3}$ of the image. Below a certain gradient value, all pixels were put in the black segment. Above a certain gradient value, all pixels were put in the white segment. Between these values, a pixel's classification is determined by whether or not it is within the hexagon.

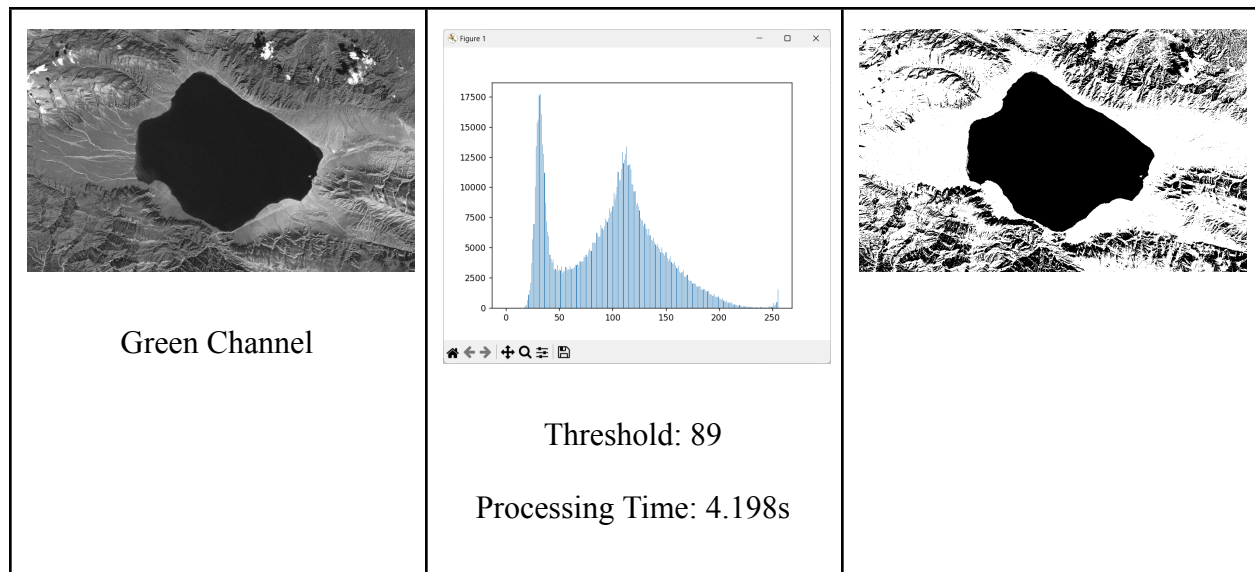


The primary challenge of the final image is that it is an RGB picture, and my implementation of Otsu's Method only considers one channel. As such, most of the picture's color information is lost in the conversion to grayscale.

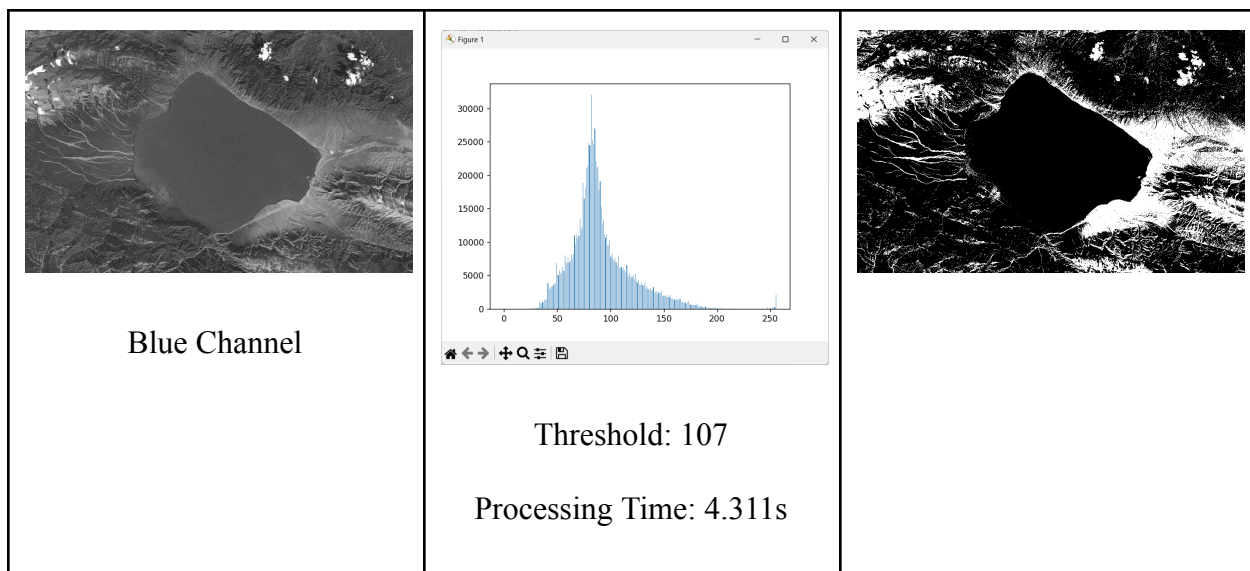


The red channel was somewhat successful at segmenting the image. The lack of red in the lake region helps to correctly classify all of its pixels, but red levels vary too much in the surrounding terrain to cleanly separate it.

This is a rare situation where the reason Otsu's Method fails is not because all binary thresholds would do the same. Binarization at ~ 40 , right after the intensity peak of the lake's pixels, would allow almost perfect segmentation.



The green channel performed the best on this image, as green values are far more consistent outside of the lake region than red. The segmentation is still not perfect, and would again be improved by picking a threshold of ~ 40 .



The blue channel fared the worst by far. This is due to the lake's pixels being separated not by their high blue intensity values, but their comparatively low red and green ones. It is only the combination of all of this information that makes the region “blue” to a human observer, which the single-channel Otsu approach cannot distinguish. Blue intensity values are consistent throughout the entire image, so no threshold value would perform meaningfully better.

Conclusions

- Outside of reading from and writing to disk, I wrote all of the code to process the images. The only exterior functions come from my “DIPLib,” where I compile work from previous assignments.

- For two-class segmentation, the trend of between class variance at each intensity threshold will always form an upside-down parabola. As such, the value will increase until it reaches a maximum and then decrease. This allowed me to establish a “base case” in my Otsu implementation where the function terminates once the values begin decreasing.
- This likely explains the difference in processing times for the different channels of the lake image. The red channel’s threshold value of 84 means that it called the recursive function 85 times before termination, whereas green and blue called it 90 and 108 times, respectively.
- The processing time difference between images is likely a result of their relative sizes.
- Otsu’s Method is useful in the automatic selection of segmentation thresholds, but this rudimentary implementation has stark limits. It will only be successful under optimal conditions, such as those of the first image. Otherwise, it is not sufficient for segmentation on its own.